

THÉORIE DES PROGRAMMES

Cours de préparation — S7 M1 Informatique (ILIA)

Prouver qu'un programme fait ce qu'il doit faire : sémantique, triplets de Hoare, invariants de boucle, terminaison, induction et limites de la vérification formelle — avec exercices corrigés.

Sommaire

1. Sémantique des programmes
2. Preuve de correction : triplets de Hoare
3. Invariants de boucle
4. Terminaison des programmes
5. Récursivité et induction
6. Limites de la vérification formelle
7. Exercices récapitulatifs

Chapitre 1 — Sémantique des programmes

La **théorie des programmes** s'intéresse à définir précisément ce que « signifie » exécuter un programme, et à en démontrer mathématiquement les propriétés (correction, terminaison), plutôt que de se fier uniquement aux tests.

1.1 Sémantique opérationnelle

Elle décrit le sens d'un programme par la façon dont son exécution transforme un **état** (une association variable $\rightarrow$ valeur) étape par étape, via des règles de transition. C'est proche de la façon dont fonctionne réellement un interpréteur.

```
État avant : {x: 3, y: 5}
Instruction : x := x + y
État après : {x: 8, y: 5}
```

1.2 Sémantique dénotationnelle

Elle associe directement à chaque programme une **fonction mathématique** qui transforme un état d'entrée en état de sortie, sans décrire les étapes intermédiaires — utile pour raisonner sur le résultat final indépendamment du détail d'exécution.

Différence clé : la sémantique opérationnelle décrit le *comment* (les étapes), la sémantique dénotationnelle décrit le *quoi* (la fonction globale calculée). Les deux coexistent et se complètent selon ce qu'on veut prouver.

Exercice 1 — Sémantique opérationnelle

Donnez l'état après exécution de la séquence d'instructions suivante, en partant de $\{x: 2, y: 0\}$:

$y := x * 2$; $x := y + 1$;

Correction

Après $y := x * 2$: $\{x: 2, y: 4\}$.

Après $x := y + 1$: $\{x: 5, y: 4\}$.

État final : $\{x: 5, y: 4\}$.

Chapitre 2 — Preuve de correction : triplets de Hoare

La **logique de Hoare** fournit un cadre formel pour prouver qu'un programme est correct par rapport à une spécification, à l'aide de **triplets** notés $\{P\} C \{Q\}$.



$\{P\} C \{Q\}$: si P est vraie avant C , et que C termine, alors Q est vraie après

Figure 2.1 — Un triplet de Hoare : précondition, programme, postcondition

$\{P\} C \{Q\}$: si la précondition P est vraie avant l'exécution de C , et que C termine, alors la postcondition Q sera vraie après. On parle de *correction partielle* (on ne garantit pas que C termine, juste que **si** il termine, le résultat est correct).

2.1 Règle de l'affectation

Règle de base : $\{Q[x/e]\} x := e \{Q\}$, où $Q[x/e]$ signifie « Q dans lequel on remplace toutes les occurrences de x par e ».

Exemple : $\{y + 1 > 0\} x := y + 1 \{x > 0\}$
On part de la postcondition $\{x > 0\}$, on remplace x par $(y+1)$ pour obtenir la précondition correcte : $\{y + 1 > 0\}$.

2.2 Règle de composition séquentielle

Si $\{P\} C1 \{R\}$ et $\{R\} C2 \{Q\}$, alors $\{P\} C1; C2 \{Q\}$. On enchaîne les preuves instruction par instruction, en utilisant le résultat intermédiaire R comme pont entre les deux.

2.3 Règle de la conditionnelle

Si $\{P \wedge B\} C1 \{Q\}$ et $\{P \wedge \neg B\} C2 \{Q\}$, alors $\{P\} \text{if } B \text{ then } C1 \text{ else } C2 \{Q\}$. Il faut prouver que Q est atteint dans les deux branches possibles.

Exercice 2 — Appliquer la règle d'affectation

Trouvez la précondition P la plus générale telle que $\{P\} x := x + 3 \{x > 10\}$ soit un triplet de Hoare valide.

Correction

On applique la règle d'affectation : on remplace x par $(x+3)$ dans la postcondition $\{x > 10\}$, ce qui donne $\{x + 3 > 10\}$, soit après simplification **$\{x > 7\}$** .

Chapitre 3 — Invariants de boucle

Prouver la correction d'une boucle est plus délicat qu'une séquence d'instructions car le nombre d'itérations n'est pas fixé à l'avance. On utilise un **invariant de boucle** : une propriété qui reste vraie avant chaque itération.



L'invariant I reste vrai à chaque étape ; combiné à la condition de sortie, il prouve le résultat final

Figure 3.1 — L'invariant I reste vrai à chaque étape de la boucle

3.1 Règle de la boucle while

Si $\{I \wedge B\} C \{I\}$, alors $\{I\}$ while B do C $\{I \wedge \neg B\}$. Autrement dit : si l'invariant I est préservé par chaque tour de boucle (tant que la condition B est vraie), alors à la sortie de la boucle, I est toujours vrai **et** B est devenue fausse.

3.2 Exemple : somme des n premiers entiers

```
s := 0; i := 1;
while (i <= n) {
  s := s + i;
  i := i + 1;
}
// on veut prouver : s = n*(n+1)/2
```

Invariant proposé : $I \equiv s = (i-1)*i/2$ (la somme des entiers de 1 à i-1). Il faut vérifier trois choses :

- **Initialisation** : avant la boucle, $s=0$ et $i=1$, donc I devient $0 = 0*1/2 = 0$. Vrai.
- **Préservation** : si I est vrai avant un tour, il reste vrai après (à vérifier par calcul algébrique en remplaçant s et i par leurs nouvelles valeurs).
- **Conclusion à la sortie** : à la sortie, $i > n$ donc $i = n+1$ (si on avance de 1 en 1), et I donne $s = n*(n+1)/2$, exactement le résultat souhaité.

Exercice 3 — Trouver un invariant

Pour la boucle suivante calculant une puissance :

```
p := 1; i := 0; while (i < n) { p := p * a; i := i + 1; }
```

Proposez un invariant de boucle liant p, a et i, puis vérifiez qu'il est vrai à l'initialisation.

Correction

Invariant proposé : $I \equiv p = a^i$. À l'initialisation, $p=1$ et $i=0$, donc I devient $1 = a^0 = 1$ — vrai pour tout $a \neq 0$. À chaque tour, p devient $p*a$ et i devient $i+1$, donc le nouveau p vaut $a^i * a = a^{(i+1)}$, ce qui correspond bien à la nouvelle valeur de i — l'invariant est préservé. À la sortie, $i = n$, donc I donne $p = a^n$, le résultat attendu.

Chapitre 4 — Terminaison des programmes

La correction partielle (chapitre 2-3) ne garantit pas qu'un programme se termine — une boucle infinie satisfait n'importe quelle postcondition « par vide ». Pour prouver la **correction totale**, il faut en plus démontrer la terminaison.

4.1 Variant de boucle

Un **variant** est une expression entière qui :

- reste toujours **positive ou nulle** tant que la boucle continue,
- **strictement décroît** à chaque itération.

Comme il n'existe pas de suite infinie strictement décroissante d'entiers naturels, l'existence d'un variant garantit que la boucle se termine en un nombre fini d'itérations.

4.2 Exemple

```
while (i < n) { ... i := i + 1; }
```

Variant naturel : $V = n - i$. Il est positif tant que $i < n$ (condition de boucle), et décroît strictement de 1 à chaque tour puisque i augmente de 1. La boucle termine donc en au plus $(n - i_{\text{initial}})$ itérations.

Exercice 4 — Trouver un variant

Pour la boucle suivante (recherche dichotomique simplifiée) :

```
while (bas < haut) { milieu := (bas+haut)/2; if (...) bas := milieu + 1; else  
haut := milieu; }
```

Proposez un variant de boucle, et expliquez pourquoi il garantit la terminaison.

Correction

Variant proposé : $V = \text{haut} - \text{bas}$. Il reste positif tant que $\text{bas} < \text{haut}$ (condition de la boucle). À chaque itération, l'intervalle $[\text{bas}, \text{haut}]$ est strictement réduit (soit bas augmente vers $\text{milieu}+1$, soit haut diminue vers milieu), donc V décroît strictement à chaque tour. Comme V est un entier positif qui décroît strictement, la boucle se termine en un nombre fini d'itérations.

Chapitre 5 — Récursivité et induction

Pour une fonction récursive, on ne prouve pas la correction avec un invariant de boucle mais avec un **raisonnement par induction**, très proche des preuves par récurrence vues en algorithmique avancée.

5.1 Principe général

- **Cas de base** : prouver que la fonction est correcte pour le(s) cas d'arrêt de la récursion.
- **Hypothèse d'induction** : supposer que la fonction est correcte pour un appel récursif de « taille » strictement inférieure.
- **Cas inductif** : prouver que si les appels récursifs sont corrects, alors le résultat global l'est aussi.

5.2 Exemple : factorielle

```
int fact(int n) {  
    if (n == 0) return 1;  
    return n * fact(n - 1);  
}
```

Cas de base (n=0) : fact(0) renvoie 1, qui correspond bien à $0! = 1$.

Hypothèse d'induction : on suppose fact(n-1) renvoie correctement $(n-1)!$.

Cas inductif : fact(n) renvoie $n * \text{fact}(n-1) = n * (n-1)! = n!$, ce qui est exactement le résultat attendu. La fonction est donc correcte pour tout $n \geq 0$.

Terminaison des fonctions récursives : il faut aussi vérifier que l'argument décroît strictement vers le cas de base à chaque appel (ici, n décroît strictement vers 0) — même principe que le variant de boucle, appliqué à la récursion.

Exercice 5 — Preuve par induction

Pour la fonction suivante calculant la somme des n premiers entiers :

```
int somme(int n) { if (n == 0) return 0; return n + somme(n - 1); }
```

Rédigez la preuve par induction que $\text{somme}(n) = n*(n+1)/2$ pour tout $n \geq 0$.

Correction

Cas de base (n=0) : somme(0) renvoie 0, et $0*(0+1)/2 = 0$. Vrai.

Hypothèse d'induction : on suppose $\text{somme}(n-1) = (n-1)*n/2$.

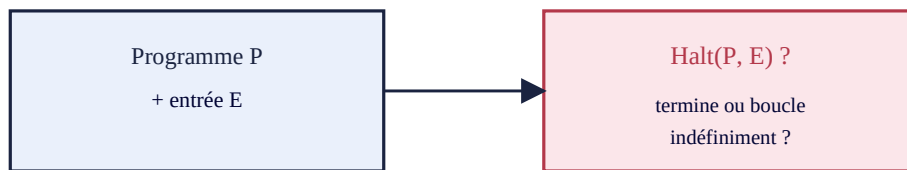
Cas inductif : $\text{somme}(n) = n + \text{somme}(n-1) = n + (n-1)*n/2 = (2n + n^2 - n)/2 = (n^2 + n)/2 = n*(n+1)/2$, exactement le résultat attendu. La propriété est donc vraie pour tout $n \geq 0$.

Chapitre 6 — Limites de la vérification formelle

Prouver un programme correct est puissant, mais pas toujours possible de façon automatique. Ce chapitre situe les limites théoriques de cette démarche.

6.1 Le problème de l'arrêt (halting problem)

Alan Turing a démontré en 1936 qu'il n'existe **aucun algorithme général** capable de déterminer, pour n'importe quel programme P et n'importe quelle entrée E , si $P(E)$ va se terminer ou boucler indéfiniment.



Aucun algorithme général ne peut répondre correctement pour tout (P, E)

Figure 6.1 — Le problème de l'arrêt : indécidable en général

Conséquence pratique : on ne peut pas construire un outil qui prouve automatiquement la terminaison de *n'importe quel* programme. En revanche, on peut souvent le prouver pour une classe restreinte de programmes (via un variant explicite, comme au chapitre 4).

6.2 Model checking

Le **model checking** est une technique de vérification automatique qui explore exhaustivement tous les états atteignables d'un système (souvent fini) pour vérifier qu'une propriété donnée est toujours respectée. Contrairement à la preuve manuelle par invariants, c'est automatisable, mais limité par l'explosion combinatoire du nombre d'états à explorer pour des systèmes complexes.

6.3 Vérification vs test

Approche	Garantie apportée
Test	Vérifie le comportement sur un nombre fini de cas précis — ne prouve rien pour les cas non testés
Preuve formelle (Hoare, invariants)	Garantit la correction pour <i>tous</i> les cas possibles, mais demande un effort manuel important
Model checking	Vérification automatique exhaustive, mais limitée aux systèmes de taille raisonnable

Exercice 6 — Décidabilité

Un enseignant affirme avoir écrit un programme qui détecte automatiquement, pour n'importe quel code source fourni en entrée, s'il contient une boucle infinie. Que peut-on en conclure à la lumière du problème de l'arrêt ?

Correction

D'après le théorème de Turing sur le problème de l'arrêt, un tel programme **ne peut pas exister en toute généralité** : il est prouvé qu'aucun algorithme ne peut décider, pour absolument tout programme et toute entrée, s'il termine ou boucle indéfiniment. Soit l'outil de l'enseignant se limite en réalité à une classe restreinte de programmes (par exemple sans récursion complexe ni structures de contrôle générales), soit il peut se tromper (faux positifs ou faux négatifs) sur certains cas.

Chapitre 7 — Exercices récapitulatifs

Exercice 7 — Preuve complète d'une boucle

Soit le programme suivant, censé calculer le maximum d'un tableau `tab` de taille n ($n \geq 1$) :

```
max := tab[0]; i := 1; while (i < n) { if (tab[i] > max) max := tab[i]; i := i + 1; }
```

- a) Proposez un invariant de boucle adapté.
- b) Proposez un variant de boucle prouvant la terminaison.
- c) Vérifiez que l'invariant, combiné à la condition de sortie, donne bien le résultat souhaité.

Correction

- a) Invariant I : `max` est le plus grand élément parmi `tab[0..i-1]`.
- b) Variant $V = n - i$: positif tant que $i < n$, et décroît strictement de 1 à chaque tour puisque i augmente de 1 à chaque itération.
- c) À la sortie de la boucle, $i = n$ (condition $i < n$ devenue fausse), donc l'invariant I donne : `max` est le plus grand élément parmi `tab[0..n-1]`, c'est-à-dire le tableau entier — exactement le résultat attendu.

Exercice 8 — Question ouverte

Expliquez pourquoi trouver un bon invariant de boucle est souvent considéré comme la partie la plus difficile d'une preuve de programme, alors que vérifier qu'un invariant proposé est correct est en général plus mécanique.

Correction

Vérifier qu'un invariant proposé est préservé par la boucle est un exercice relativement mécanique : on applique les règles de la logique de Hoare à chaque instruction du corps de boucle et on vérifie l'implication logique attendue. En revanche, **trouver** l'invariant demande de comprendre en profondeur ce que fait réellement la boucle, d'anticiper une propriété assez forte pour permettre de conclure le résultat final à la sortie, mais assez générale pour être réellement préservée à chaque itération — un équilibre qui ne se déduit pas automatiquement du code et demande de l'intuition et de l'expérience.